Bil495 - Innovative Computer Application
Lord of the Strings

Submitted By

Aylin Doğan 221101058

Emir Yücedağ 221101077

Kerem Kazandır 221101007

Zeynep Yağmur Bozdağ 221101078

Department of Computer Engineering
TOBB University of Economics & Technology

Spring 2025

TABLE OF CONTENTS

ABSTRACT

The Gather Up project addresses the critical issue of coordinating social activities in a time-constrained environment by developing a Real-Time Availability Based Social Interaction Platform. Existing social and event-finding applications fail to synchronize users based on their immediate, real-time availability, leading to missed opportunities for spontaneous meetups and inefficient activity planning.
Gather Up proposes a solution through a Modular Monolith Architecture that hosts a proprietary Matching Engine (Core Logic). This engine utilizes a multi-factor scoring algorithm, combining user availability, interests, and proximity to generate highly personalized event recommendations within a tight latency constraint (under 100 milliseconds).
The platform prioritizes user safety and accountability. This is achieved through a dynamic Safety Score mechanism, which updates a user's reliability rating based on post-event feedback and attendance history. The system is implemented using a React Native mobile client and a Node.js/Express.js backend utilizing MySQL for persistence and WebSockets for real-time chat. The entire system is built with strict adherence to GDPR and KVKK standards, focusing on data privacy and ethical compliance.
Gather Up's goal is to increase social engagement, improve time management, and foster meaningful connections among university students and young professionals by making activity planning intuitive, safe, and instantaneous.

1. INTRODUCTION

1.1. Project Purpose

In today’s fast-paced world, people struggle to manage their free time effectively. Between work, classes, and daily responsibilities, individuals who want to engage in social activities often face uncertainty about who to meet, where, and when. This leads to many wasted opportunities for social interaction and personal development, as people either spend their free time unproductively or fail to connect with others who share similar interests.

Existing social media and event platforms primarily focus on broad networking or large-scale events, but they do not take into account users’ real-time availability. As a result, they are ineffective in facilitating spontaneous meetups or personalized activity planning. Even when people try to plan events in advance, differences in schedules and limited visibility into others’ availability often make coordination difficult. This lack of synchronization discourages people from initiating activities, ultimately reducing opportunities for social engagement, collaboration, and building meaningful connections.

A solution is needed that can dynamically match individuals based on their free time and preferences, enabling them to discover, create, and join activities that fit naturally into their daily lives.

1.2. Project Scope

Our product will make an impact as a brand new social media type. We will increase the competitiveness and market growth in this field by making unknown brands in the entertainment industry more accessible through our application.
On the other hand our app has no concern about replacing other social media platforms. It will have its own uniqueness.
For the larger scale our app can have some legal issues and could have some security problems because of the physical meetings. So we are making it for students because It's easier to authenticate via their emails.

1.3. Definitions, Acronyms, and Abbreviations

UI: User Interface
UX: User Experience
API: Application Programming Interface
GDPR: General Data Protection Regulation
KVKK: Kişisel Verileri Koruma Kanunu

1.4. References

We will update this part when related works are used.

1.5. Overview

2. part is about general description you can find general summarization of the app.
3. part is about more detailed descriptions and requirements which app have.
4. part is about data storage and dataset which will be used for recommendation.
5. part is on which systems, libraries and frameworks we use.
6. part is about which test methods we use, their impact on quality, and which systems we use to ensure quality.
7. part contains information about the expected construction process of the project and indicates who will be responsible for which part.
8. part helps the user to briefly understand what to expect from the application.
9. part addresses the social, economic and legal effects and restrictions of the app.

2. OVERALL DESCRİPTİON

2.1. Product Perspective

Gather Up is a **standalone** social interaction platform. It is not an extension of any existing campus or external system but is designed for future integration.

1. **Dependencies:**

- **External Location Service:** Required for mapping event locations and calculating user proximity (e.g., Google Maps API).
- **External Notification Service (FCM):** Directly dependent for the push notification functionality.
- **University Infrastructure (Optional):** Future dependency for student identity verification or academic calendar synchronization (not assumed for the initial launch).

2.2. Product Functions

A high-level summary of the system capabilities:

Real-Time Matching and Recommendation: Provides instantaneous event suggestions based on the user's availability calendar, interests, and location.

Event Management: Allows users to create, request participation in, and manage the lifecycle of events.

Safety Score Management: Maintains a dynamic scoring system that measures the reliability of each user based on participant feedback and attendance history.

Real-Time Chat: Facilitates instant messaging among accepted event participants.

Notifications: Sends immediate alerts to users for event approvals, new chat messages, and feedback prompts.

2.3. User Characteristics

Characteristic	Description
Target Audience	Registered students within a university or college environment.
Technical Expertise	Moderate (Familiarity with using modern mobile applications).
Motivation	Seeking to socialize and participate in organized events with peers who share similar interests in a safe, controlled environment.
Accessibility	Must comply with standard accessibility guidelines for both iOS and Android platforms.

2.4. Constraints

Technical Constraints: The platform will only support **React Native** based mobile clients initially. A dedicated web interface is out of scope for this version.

Operational Constraints:

All feedback must be anonymous to the public but must be included in the **Safety Score** calculation to ensure integrity.

The Matching Engine must return a response time of under **100 milliseconds**.

Legal/Ethical Constraints:

Handling of user data must comply with relevant data protection acts (e.g., GDPR/KVKK).

User location data must only be used for matching purposes and only with the user's explicit consent.

2.5. Assumptions and Dependencies

Assumptions:

- Target users will keep Push Notification permissions active on their devices for continuous platform functionality.
- The APIs of External Location Services (e.g., Google Maps) will offer predictable performance and pricing models.

Dependencies:

- Continuous and high availability of the Node.js Runtime and the MySQL Database Server.
- The existence of a stable internet connection on user devices.

3. SPECİFİC REQUİREMENTS

This section details every functional and non-functional requirement necessary for the system's successful operation.**External Interface Requirements**

3.1.1. User Interfaces

Dashboard: The main screen shall present a dynamically updated, scrollable list of event recommendations based on the user's current availability.

UX: The interface shall be designed for easy one-handed operation and should support light/dark mode options.

Chat Interface: The interface shall support real-time message sending, viewing, and message status display (sent/read).

3.1.2. Hardware Interfaces

Cloud Infrastructure: The system shall be hosted on a robust cloud infrastructure (e.g., AWS/Azure/GCP) and must support horizontal scaling.

GPS/Location: The mobile client shall be able to access the device's **GPS hardware** (with user permission) to retrieve real-time location coordinates.

3.1.3. Software Interfaces

Location APIs: The system shall use standard RESTful APIs for retrieving location data and rendering maps.

Database Driver: Node.js shall utilize an ORM (Object-Relational Mapping) library (e.g., Sequelize or TypeORM) for secure communication with MySQL.

3.1.4. Communication Interfaces

API Communication: All API calls shall use the **JSON** format over **HTTPS**.

Real-Time Communication: The **WebSockets** protocol shall be used for the chat module to ensure real-time, low-latency messaging.

3.2. Functional Requirements

ID	Requirement Description (The system shall...)
FR 2.1	The system shall present event recommendations within **100ms** based on the user's current availability calendar, interests, and location.
FR 2.2	The system shall instantly notify potential participants (those who meet the matching criteria) when a new event is created.

ID	Requirement Description (The system shall...)
FR 2.3	The system shall trigger a prompt for mutual feedback and rating from all participants and the organizer once an event is completed.
FR 2.4	The system shall dynamically update each user's **Safety Score** based on the aggregated post-event feedback data.
FR 2.5	The system shall automatically suspend a user's ability to create or join events if their Safety Score falls below a predefined critical threshold.
FR 2.6	The system shall create a real-time (WebSocket-based) chat room for accepted participants of an event.
FR 2.7	The system shall provide functions for users to save, edit, and delete their availability calendar entries.
FR 2.8	The system shall never store sensitive data (passwords, location history) in plaintext.

3.3. Performance Requirements

Recommendation Latency: The event recommendations list from the Matching Engine shall be processed on the server side in a maximum of **100 milliseconds**.

API Response Time: All core API calls, excluding initial authentication, shall respond in less than **200 milliseconds** on average.

Concurrency: The system shall be able to support at least **500 concurrent active users** without performance degradation.

3.4. Design Constraints

Software Standards: The backend code must adhere to current security and performance best practices within the Node.js ecosystem (e.g., Express.js security guidelines).

Data Constraint: User location data shall be purged from the temporary cache within a maximum of 24 hours after matching is complete.

3.5. Software System Attributes

3.5.1. Reliability

System Uptime: The expected operational uptime for the application server shall be **99.9%** (approximately 8 hours of downtime per year).

Fault Tolerance: The database server shall be configured to support automatic **failover** mechanisms.

3.5.2. Availability

The system shall be operationally accessible **24/7** (24 hours a day, 7 days a week).

Backup and recovery procedures shall be planned so as not to interrupt user access.

3.5.3. Security

Authentication: All users shall use a secure, token-based authentication scheme (e.g., **JWT**) managed by the Authentication Module.

Data Encryption: Passwords in the database shall be encrypted using **one-way hashing** algorithms (e.g., bcrypt). Client-server communication shall be encrypted using **SSL/TLS**.

3.5.4. Maintainability

Modularity: The application layer must maintain a **Modular Monolith** structure, allowing each module to be updated or replaced independently.

Logging: A centralized and structured logging system (e.g., ELK Stack) shall be implemented for critical operations, errors, and security events.

3.5.5. Portability

All server code shall be packaged in **Docker** containers for easy deployment and portability.

The mobile codebase shall be easily portable to both iOS and Android due to the use of **React Native**.

3.6. Other Requirements

Performance Monitoring: A monitoring tool (e.g., Prometheus) shall be integrated to track server CPU, memory usage, and API response times.

Test Coverage: Critical business logic (Matching Engine and Safety Score calculation) must have high unit and integration test coverage (Minimum **80%**).

4. DATASET

This section provides details about the dataset(s) used in the project, including their sources, characteristics, and relevance to the problem being addressed. If the project involves collecting, cleaning, or preprocessing data, describe those processes here.

4.1. Dataset Description

We did not receive any data yet. We are looking for suitable datasets. Furthermore we will make a special dataset for this app with existing datasets and our producing data.

4.2. Data Collection (If Applicable)

4.3. Data Storage

At this stage, we are currently considering a MySQL and JavaScript based solution, as the data size of the project is not very high.

4.4. Data Preprocessing

We are planning to create a customized dataset using the parts of the datasets we find online that are useful to us and the data we produce ourselves. After the data is produced, we consider applying the necessary pre-processing steps depending on the situation.

4.5. Data Limitations and Assumptions (If Applicable)

5. SYSTEM ARCHİTECTURE AND TECHNİCAL DESİGN

This section details the methods and processes used to develop the project. It covers requirement gathering, system design, implementation strategies, and the selection of tools and technologies.

5.1. Architectural Design

The Gather Up platform utilizes a **Modular Monolith Architecture** hosted within a cloud-based **Application Server (Node.js/Express.js Runtime)**. This structure balances the speed of a single deployment unit with the organizational benefits of microservices, aligning well with the rapid iteration needs of a university-based social platform. The system is layered into three primary tiers: Presentation, Application, and Persistence.

5.1.1. Final System Architecture Overview

The system is visualized as a unified structure emphasizing the data flow between core components.

- **Presentation Layer:** Handled by the **Mobile Client (React Native)**.
- **Application Layer:** Contains the **Application Server** hosting the business logic (Matching Engine, Event, and User Management Modules).
- **Persistence Layer:** Managed by the **MySQL Database**.

5.1.2. Component Deployment and Interaction (Deployment View)

The system's physical deployment involves four key nodes interacting over secure channels:

- **Application Server Database Server (MySQL):** Communicates via **TCP/IP (secure DB connection)** for persistent data operations.
- **Application Server Mobile Client:** Communicates via **HTTPS (API calls)** for general operations and **WebSocket** for real-time chat functionality.
- **Application Server FCM Service:** Communicates via **HTTPS** to send push notifications to users.

5.1.3. Internal Modular Design (Application Layer Detail)

The **Application Server** is structured as a **Modular Monolith**, where key functionalities are encapsulated as independent modules that communicate internally:

- **Authentication Module:** Acts as the gateway, validating all incoming requests.
- **Matching Engine (Core Logic):** The central algorithm, receiving **availability data** from the **Real-Time Availability Module** and **preferences/feedback** from the **User Management Module** to generate event recommendations.
- **Chat Module (WebSocket Server):** Handles real-time communication between accepted event participants.

- **Event Management Module:** Controls the lifecycle and state of all events (Created, Open, Active, Completed, Archived).

5.2. Technology Stack

The following frameworks, libraries, and tools were selected based on their performance, scalability, and ease of development:

Category	Technology	Purpose
Backend Runtime	**Node.js**	Execution of server-side code.
Backend Framework	**Express.js**	Creating RESTful API endpoints and managing server logic.
Mobile Development	**React Native**	Developing the mobile application for iOS and Android using a single codebase.
Database	**MySQL**	Persistent and secure storage of relational data.
Real-Time Communication	**WebSockets**	Low-latency, bi-directional communication for the chat module.
Notifications	**Firebase Cloud Messaging (FCM)**	External, reliable push notification service.
AI/ML Logic	(If Necessary) Python Libraries (NumPy, SciPy)	For complex similarity score calculations within the Matching Engine.

5.3. Implementation Details

5.3.1. Programming Logic: The Matching Engine

The core value of the platform relies on the **Matching Engine** logic. This logic is implemented as a scoring algorithm that combines three primary factors:

1. **Availability Overlap:** Binary score (1 or 0) indicating if the event's time falls within the user's available time slot.
2. **Interest Similarity Score:** Calculated using a weighted average or vector similarity based on user and event tags/preferences.
3. **Proximity/Location Weight:** A decay function applied based on the distance between the user's current location and the event location, prioritizing closer events.

5.3.2. State Management Implementation

The system rigorously enforces lifecycle management using State Machine logic to ensure data integrity, especially for events and user relationships:

- **Event Lifecycle State Machine:** Manages transitions from **Created** $\rightarrow$ **Open/Ready** $\rightarrow$ **Active** $\rightarrow$ **Completed** $\rightarrow$ **Archived**. The **Completed** state triggers the mandatory **Safety Score update** process.
- **User Relationship State Machine:** Manages friendship/interaction states (**Pending, Accepted, Blocked**).

5.3.3. Security and Ethical Implementation

A key focus is placed on the ethical and safety aspects:

- **Authentication:** All user interaction relies on secure, token-based authentication (JWT/OAuth) managed by the **Authentication Module**.
- **Safety Score:** The **User Management & Feedback Module** implements the **Safety Score** as a dynamic reliability rating based on post-event feedback (ratings, attendance confirmation). This score influences the matching priority and acts as a preventative measure against misuse.

6. TESTİNG & QUALİTY ASSURANCE

This section defines how the software reliability and quality of the Gather Up platform will be ensured throughout the project, with a core focus on real-time matching algorithm performance and user data security.

6.1. Software Quality Assurance Plan (SQAP)

The primary goal of the Quality Assurance Plan is to verify that the software meets both functional requirements (specifically matching accuracy and event creation) and non-functional requirements (real-time performance and data security). Our approach adopts an Agile development methodology, integrating Quality Assurance (QA) into continuous, iterative cycles. Functional and regression testing will be conducted at the end of every sprint and supported by continuous integration (CI) processes. For establishing relevant standards, the ISO/IEC 25010 (SQuaRE) standard will be utilized, focusing on the sub-characteristics of Usability, Performance Efficiency, and Security. Additionally, fundamental clean code principles and established secure coding best practices will serve as the primary guidelines for maintaining code quality and integrity.

6.2. Verification and Validation (V&V)

A multi-layered testing approach will be implemented to ensure the platform correctly implements all requirements and meets user needs.

- **Testing Approach:** Testing will progress through four distinct levels. Unit Tests will verify the correctness of the smallest code units, including algorithm logic, data processing functions, and critical business rules (like availability checks). Integration Tests will focus on data flow between the matching algorithm and the database, and the secure integration with third-party services (e.g., mapping and geolocation Open APIs). System Tests will evaluate the entire system as a cohesive whole, including performance tests to measure the real-time matching speed and security tests to confirm authorization and data protection mechanisms. Finally, Acceptance Tests (UAT) will involve the target users (university students, young professionals) to validate the application's ease of use and adherence to functional goals.
- **Coverage Metrics:** Quality will be evaluated through quantitative metrics. Requirement-Based Testing will track the completion percentage of test scenarios corresponding to each core objective (smart matching, event creation, feedback system). Fault Density (the number of defects found per module, such as the matching module) will be utilized to gauge the underlying quality of the code base.
- **Security and Compliance:** Security is a critical V&V area. Compliance with GDPR and local data protection laws will be ensured through regular reviews of data processing flows and user consent mechanisms. Furthermore, adherence to secure coding practices is mandatory to mitigate common vulnerabilities identified in frameworks like the OWASP Top 10.

6.3. Configuration and Change Management

- **Version Control:** The entire codebase and project assets will be managed using Git. Transitions between development, feature, and main branches will be strictly governed by a controlled Git workflow (e.g., Gitflow or GitHub Flow).

- **Change Tracking and CI/CD:** All change requests and reported issues will be meticulously tracked using a ticket/issue tracking system (e.g., GitHub Issues). No changes will be merged into the main codebase without passing through Continuous Integration/Continuous Deployment (CI/CD) pipelines that automatically execute all necessary tests. Mandatory peer code reviews will be enforced for every code integration to ensure quality and adherence to standards.

6.4. Risk and Defect Management

Software defects will be systematically recorded, managed, and resolved using the designated issue tracking system.

- **Tracking and Classification:** Every identified defect will be classified based on its severity (Critical, High, Medium, Low) and priority. Critical (e.g., data breach or a fundamental matching function crash) and High priority defects must be addressed and resolved before the start of the next development sprint.
- **Resolution and Verification:** Once a defect is fixed by the development team, it must be thoroughly verified and approved by the QA team before the issue can be formally closed, ensuring the fix did not introduce new regressions.

6.5. Product Evaluation and Acceptance

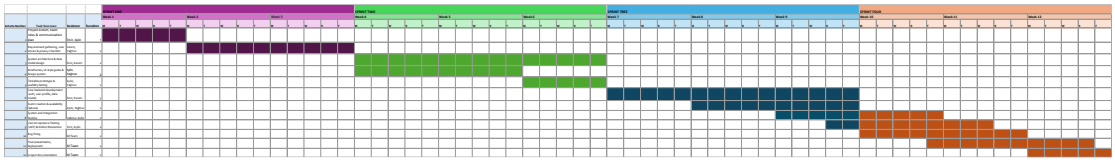
The final acceptance of the software is contingent upon proving the successful achievement of the MVP goals and meeting defined critical performance thresholds.

- **Acceptance Criteria:** Acceptance requires the platform to meet specific criteria across multiple dimensions: Functionality (all core features—availability input, smart matching, event creation, participation, and feedback—must be 100% operational according to requirements); Performance (the real-time matching algorithm must return results within a defined maximum response time, such as under 2 seconds); Security (successful security and penetration test results with proven data privacy); and Usability (the user interface must pass a predefined satisfaction threshold established during UAT).
- **Error Handling:** Acceptance is contingent upon zero open defects classified as Critical or High severity. The system's robustness in handling expected errors and exceptions must be documented and verified.
- **Review Processes:** To ensure software readiness, peer reviews are mandatory during the development phase. Furthermore, a final audit will be conducted at the end of the project to verify compliance with all legal, ethical, and lifecycle requirements, providing final validation of the product’s quality and fitness for deployment.

7. PROJECT MANAGEMENT AND RİSK ANALYSİS

7.1. Project Plan

Gantt Chart for 12 Weeks.



7.2. Task Distribution

Describe how the workload was divided among team members. Include a table or bullet points specifying which team member worked on each part of the project. If applicable, mention any pair programming or collaborative tasks. You can add other rows if exists.

Task	Team Member(s)	Description
UI/UX Design	Aylin, Yağmur	Created wireframes and designed the user interface.
Backend Development	Emir, Kerem	Implemented the database and API logic.
Frontend Development	Aylin, Yağmur	Developed the client-side functionality.
Testing & Debugging	Yağmur	Conducted unit tests and fixed reported bugs.
AI Development	Kerem	Developed Recommendation System for Events.
Documentation & Report Writing	All Team	Compiled the final project report.

7.3. Risk Analysis and Mitigation Strategies

We are aiming to make our own dataset so while making it we will try to lower bias.
Because of we did not choose which models will be used, we do not have any idea of explainability.
Our model will use data from users and events with that we will modify our model.
And we will implement all of this in accordance with KVKK and GDPR.

8. USE CASES AND USER STORİES

8.1. Target Audience

Students and young professionals who want to quickly find out what activities they can participate in during their free time.

8.2. User Stories

As a student, I want to organize an event with my friends.
As a parent, I want to know if my child is using a secure app.
As a young professional, I want to participate in an event after my job.
As a shop owner, I want more people to come into my shop and use it.
As a student club president, I want more people to be aware of the activities we do as a club.
As a university administrator, I want to track the number of active student events so that I can allocate resources effectively.
As a new student, I want to discover nearby social or academic events so that I can make new connections.
As an event organizer, I want to collect feedback from participants so that I can improve future events.

9. SOCİAL, ENVİRONMENTAL, AND LEGAL IMPACT

9.1. Societal Benefits

Our event matching system will help to manage time management. With that people will have a more organised life.

9.2. Economic Constraints

We will use design patterns for coding with that we are aiming to increase code efficiency and performance efficiency
For the app we will use a cloud provider (like Google Cloud or Azure) that is publicly committed to 100% renewable energy for their data centers..

9.3. Legal and Ethical Compliance

We will use the data of our users and activities according to GDPR and KVKK standards.

REFERENCES

List all the sources cited throughout the report using the **IEEE citation format**. This section should include books, research papers, online articles, datasets, and any other references used for literature review, methodology, or implementation.
What to Include in This Section:

- Follow **IEEE citation style**, which uses **numerical referencing** (e.g., [1], [2]).
- Ensure all references are **properly formatted** with full details (authors, title, publication, year, and DOI/link if applicable).
- Cite **academic and reliable sources** instead of unverified websites.

Example reference (IEEE format):
[1] A. Smith, “Introduction to Machine Learning,” Journal of AI Research, vol. 34, no. 2, pp. 101-120, 2023.

APPENDIX

The appendix contains **supplementary materials** that are relevant to the project but not included in the main report body. It provides additional technical details, large datasets, code snippets, or extended explanations that support the content discussed in earlier sections.

What to Include in This Section:

- **Extended tables, figures, or raw data** that are too detailed for the main text.
- **Source code snippets or algorithms** (if necessary) to clarify implementation details.
- **Detailed mathematical derivations or calculations** used in the project.
- **Additional documentation or survey results** (if user testing or external data collection was conducted).

If multiple appendices are included, label them as **Appendix A, Appendix B, etc.** and refer to them in the main report as needed.